

仓颉代码片段语义检查-答辩报告

赛题名称：仓颉代码片段语义检查

竞赛：2026 年全国大学生计算机系统能力大赛——编译系统挑战赛

队伍：圆周运动 (T2026100552010674)

学校：南开大学

目录

- 仓颉代码片段语义检查-答辩报告
 - 目录
 - 评审速览：作品特色与直接证据
 - 摘要
 - 1. 问题分析与设计目标
 - 1.1 逐 token 前缀判断
 - 1.2 设计目标与范围
 - 1.3 官方材料协议差异与兼容
 - 2. 五阶段基础演进与最终版本
 - 3. 最终版本原生 C++ 架构与功能
 - 3.1 构建期与运行时数据流
 - 3.2 主要模块
 - 3.3 功能覆盖
 - 3.3.1 决赛新增 context 能力
 - 3.3.2 完整赛题功能覆盖
 - 4. 核心技术贡献与当前实现伪代码
 - 4.1 稳定词法事实与可延长词素分流
 - 4.2 前缀可续写性判定模型
 - 4.2.1 定义与状态
 - 4.2.2 不可恢复错误判定
 - 4.2.3 PartialLexeme 处理原则
 - 4.2.4 设计不变量
 - 4.2.5 复杂度与最坏情况
 - 4.3 活动语句缓存覆盖多行表达式
 - 4.4 传递泛型接口要求与 alpha-renaming
 - 4.5 决赛 context 语义加固
 - 4.6 证明携带的续延判定与决策台账
 - 4.7 逐重载调用前沿与硬提交边界
 - 4.8 单步可恢复性与 lambda twin

- 4.9 文法平坦化与标识符有限商
- 5. 重难点、修改历程与解决方案
 - 5.1 BPE 分片与仓颉词法边界
 - 5.2 合法未完成前缀的提前误报风险
 - 5.3 每 token 重复扫描与深度解析
 - 5.4 多行函数签名进入语义模型
 - 5.5 嵌套 lambda 参数可见性
 - 5.6 泛型接口传递继承与签名规范化
 - 5.7 重载方法引用歧义
 - 5.8 函数作用域与关键字边界
 - 5.9 隐藏样例覆盖评估
 - 5.10 context API 差分与样例口径校准
 - 5.11 context 结构与官方行为不一致
 - 5.12 开放表达式、合法续写与硬提交边界
 - 5.13 宽恢复策略的回退与证明化收敛
 - 5.14 长前缀规模退化与窄优化
- 6. 历史性能测试与最终平台结果
 - 6.1 实验环境与统计方法
 - 6.2 决赛最终提交平台结果
 - 6.3 五阶段历史平台性能
 - 6.4 内部原生化版本的冷进程辅助数据
 - 6.5 与类似方法和参考工作的对比
- 7. 正确性与鲁棒性测试
 - 7.1 验证方法
 - 7.2 分层验收结果
 - 7.3 代表性复现命令
- 8. 为什么选择原生 C++ 与持续语义加固
 - 8.1 团队的工程取舍
 - 8.2 加固重点降低的潜在风险
- 9. 已知边界与适用范围
- 10. 第三方来源与原创贡献声明
 - 10.1 可审计组件清单
 - 10.2 第三方能力、本队适配与独立实现边界
 - 10.3 竞赛配套 typechecker 的开发期用途与提交边界
 - 10.4 最终提交框架清单
- 11. 参考文献
- 12. 结论
- 附录 A: 初赛历史逐例平台性能数据
 - 附录 A 汇总
- 附录 B: 决赛最终提交平台数据
 - 附录 B 汇总

评审速览：作品特色与直接证据

初赛最终开发实现版本

从“可判断”到“可在线实时判断”

XGrammar 通用语法约束与本体原生 C++ 前缀语义规则协同工作，重点控制合法前缀的提前误报。

100 / 100

公开错误样例

精确命中首个不可恢复 token

8.55x / 近10x

平台平均提速

相对 XGrammar + 仓颉简单适配基线

核心技术贡献：在不完整输入中分流稳定词法事实与可延长词素，并结合缓存模型执行选择性语义探测

任意 BPE / UTF-8 分片

稳定词法 TokenEvent

PartialLexeme 保留

缓存模型选择性 Probe

决赛指标

最终记录

平台结果

63.00 / WA

平台列出的计时条目

63

条目累计 / 平均耗时

16.982s / 0.26956s

条目耗时范围

0.242s-0.315s

本作品在 XGrammar 通用语法约束能力之上，构建了面向仓颉代码片段的原生 C++ 增量前缀语义检查路径。最终版本使用 TokenEvent 固化已经确定的词法边界，使用 PartialLexeme 保存仍可延长的末尾内容；语义层复用声明快照、函数上下文和活动语句，并新增 FrontierInfo、逐重载 CallFrontier、ContinuationProof 与只读审计用的 DecisionLedger。只有“命中由离线完整 tail 复验支持的开放数组延期规则”或“在硬提交点完整淘汰全部候选”这两类规则级证据可以覆盖基线判断。

决赛最终提交得分为 63.00，判定为 WA，算术平均为 0.26956s/条。该结果说明最终版本已覆盖相当一部分容器、lambda 与泛型推导组合，但仍存在未覆盖的隐藏语义场景；初赛旧加固纯 C++ 版本的 0.27328s 与 8.55x（历史旧加固版）仅作为架构演进证据，不能与决赛 63 条记录直接换算。

本队真实贡献集中在仓颉赛题适配和增量语义实现：C++ 竞赛入口、token 查表解码、IncrementalLexer、TokenEvent/PartialLexeme、原生语义模型、以官方 context 为来源并经行为校准的运行单源加载、Array 首末属性语义、lambda twin、字段优先冲突处理、单步可恢复性、逐重载调用前沿、证明携带决策，以及差分和回归验证工具。XGrammar 通用 matcher、tiktoken 编码定义和竞赛配套 typechecker 均不属于本队原创；TVM FFI 也不进入最终生产运行路径。

摘要

赛题要求检查器逐个接收 `cl100k_base` token ID，并在每轮输入后判断当前仓颉源码前缀能否通过后续输入补全为合法程序。赛题说明文字使用 1=可续写、0=不可续写，组委会公开 checker 的实际交互协议使用 0=继续、1=首错。无参数运行的 `solution` 遵循公开 checker 协议；`--competition-output` 兼容模式只翻转输出编码，两种模式共用检查状态和首错位置。

最终版本采用“XGrammar 字符级语法约束 + 本队原生 C++ 前缀语义检查”的双层架构。提交包自带压缩 token 表、以官方 context 为来源且经团队行为校准的二进制模型、仓颉 GBNF 与未作功能性修改的 XGrammar v0.2.1 C++ core；构建过程不访问网络，运行时不依赖 Python 绑定或 TVM FFI。token ID 经本地查表还原为 UTF-8 字节，语法状态与增量词法并行推进；语义层再结合 context 模型、活动上下文、前沿分类和证明携带决策，对开放表达式保守延迟、对硬提交点执行候选穷尽判定。

最终获得 63.00 分，平台判定 WA。平台列出的 63 条计时记录累计 16.982s，平均 0.26956s/条，中位数 0.270s。

1. 问题分析与设计目标

1.1 逐 token 前缀判断

本题采用逐 token 的前缀可续写性判断。BPE token 边界与仓颉词法 token 边界相互独立，一个标识符、数值、多字符运算符、UTF-8 字符、字符串或嵌套注释可能跨越多轮输入。检查器需要在语义事实充分时尽早锁存错误，同时为仍有合法延长方式的片段保留补全空间。

例如，输入到标识符中途时，该串可能在下一轮继续构成可见名称；输入到字符串、函数参数列表或 lambda 表达式中途时，程序结构仍在形成。已经完成的 `let` 变量重新赋值、循环外 `break`、确定无法匹配候选函数的调用等情形，则可在当前轮形成不可恢复错误。

决赛实现进一步在证明覆盖层把内部前沿划分为 Alive、Dead 和 Unknown：命中由离线正向证据支持的延期规则时为 Alive，当前生产中仅开放数组元素使用这一状态；该覆盖层若要新产生 Dead，必须位于 `)`、`]`、`}`、已提交参数分隔或下一语句等硬边界，并持有完整候选集穷尽证明；普通语义错误仍由既有基线规则处理。开放表达式搜索不到后缀只能保持 Unknown，不能把“尚未证明可续写”误当作“已经证明不可续写”。因此，本题的难点不只是判断错误是否存在，更是确定首个错误已经无法被未来 token 修复的时刻。

1.2 设计目标与范围

- 每轮输入后立即输出可续写性判断，并在第一次不可恢复错误处锁存失败状态；
- 覆盖赛题涉及的增量词法、语法、声明、作用域、类型、控制流、调用、泛型、类/接口和 lambda 子集；
- 对每次覆盖基线的决定保存证据种类、规则、候选淘汰原因和决策台账；无证据时回退基线，而不是猜测；
- 分层报告历史公开回归、本地语料和决赛平台结果，不用任一内部 oracle 替代最终外部评测；
- 将竞赛热路径收敛到原生 C++ 进程，减少解释器启动、完整前缀扫描和深度解析；
- 以合法程序的全部观测前缀零误报作为与精确首错同等重要的验收标准；
- 保持第三方来源、团队修改和团队独立实现的归属清晰、可复查。

本作品面向赛题规定的仓颉语言子集。文中结论限定在公开语料、项目语料、随机生成语料、分片测试、sanitizer 测试和已记录的平台提交结果范围内。

1.3 官方材料协议差异与兼容

组委会公开的 [cangjie-fragment-checker](#) 使用逐 token stdin/stdout 交互，其 README 和 `scripts/token_interaction_test.py` 明确采用 0=当前无错、1=在本轮报告首错。公开 checker 的典型调用方式为：

```
python3 scripts/token_interaction_test.py wrong/err_arity.cj --cmd ./solution
```

公开 checker 不附加输出翻转参数，因此本项目默认启动命令和输出如下：

场景	启动方式	可续写	不可续写
公开 checker 与最终版本默认路径	./solution	0	1
赛题说明文字约定	./solution --competition-output	1	0

--competition-output 只在最终输出处翻转布尔编码，不改变 token 解码、语法状态、语义状态和首错位置。最终版本默认模式以公开 checker 的实际协议为准；若运行方采用赛题文字约定，可直接调整启动参数，无需重新实现检查算法。

2. 五阶段基础演进与最终版本

初赛文档中的五阶段记录保留为历史演进证据。XGrammar 通用语法引擎属于第三方；仓颉 GBNF、竞赛协议、token/context 适配和语义检查器为本队围绕赛题完成的工作。答辩主版本在历史第五阶段之后继续完成决赛 context 校准和前沿证明化，单列为最终版本。

阶段	版本	对应平台 条目平均 耗时	相对 历史 基线	本队实现内容
1	XGrammar + 简单适配基线	2.33746s	1.00×	完成仓颉 GBNF、token 适配和初始语义检查；每个输入 token 触发完整前缀扫描、Lark 解析和类型检查。
2	纯 Python 优化版	0.65642s	3.56×	默认热路径停用每 token Lark 深检，缓存函数、类、接口和上下文。
3	混合稳定版	0.31690s	7.38×	C++ 负责协议、解码和语法状态，Python worker 负责增量语义。
4	未加固纯 C++ 版	0.26866s	8.70×	将增量词法、类型、作用域、调用、泛型和 lambda 等热路径迁移到 C++。
5	历史旧加固纯 C++ 版	0.27328s	8.55× (历史旧加固版)	增加多行声明、嵌套 lambda、泛型接口继承、重载消歧和作用域隔离。
6	最终版本	0.26956s (平台列出的 63 条记录)	不与历史 倍数 混算	以官方 context 为来源、经行为校准的提交 context.json 为运行时单一数据源，补齐 Array 首末属性、lambda twin、字段优先冲突、单步可恢复性、数组/调用硬边界、逐重载前沿和证明携带决策；同时加入块语句文法平坦化与标识符有限商，控制长前缀状态增长。

决赛还处理了两类不直接增加短例分数、但决定规模稳健性的退化：将 block 语句表从右递归改写为等价平坦闭包；在 XGrammar 前对普通 ASCII 标识符构造“文法可观察的有限商”，仅向 matcher 提交仍可能影响字面量关键字判断的代表字符。专门规模测试中，前者把 300 个局部声明的运行从超过 35s 降到约 0.45s，后者把约 4 KiB 长标识符从超过 30s 且未完成降到 22–33ms。二者在官方短例汇总上的收益分别只有约 3.896% 和 0.706%，因此本文将其表述为长前缀稳健性优化，而非普遍短例加速。历史性能记录仍只说明原生化演进；决赛数据与初赛 50 例样本不同，本文不据此重算倍数。

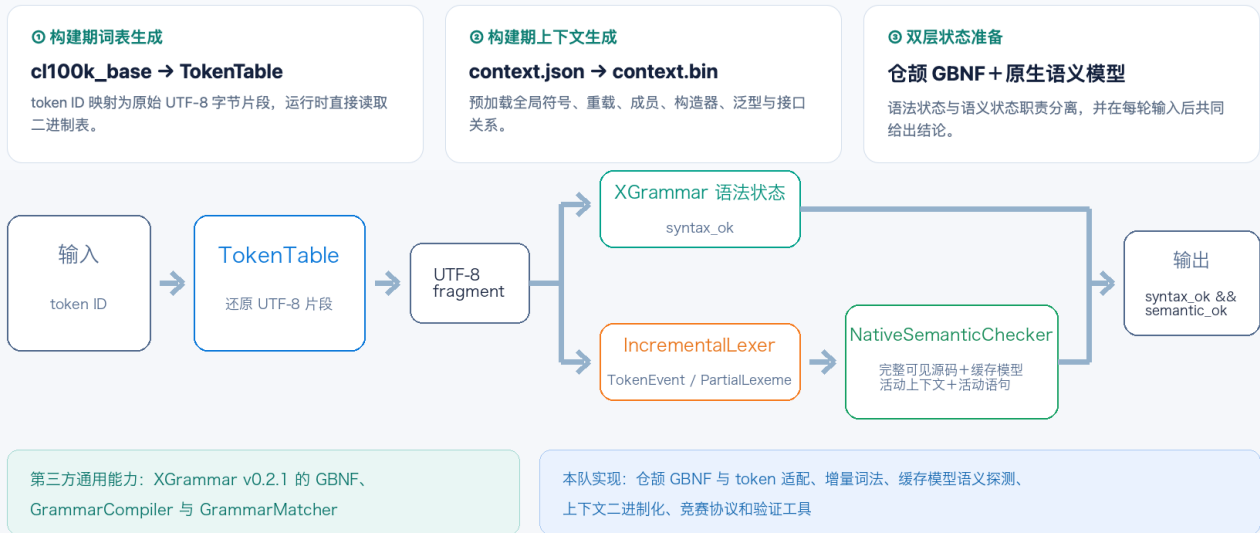
3. 最终版本原生 C++ 架构与功能

3.1 构建期与运行时数据流

系统全流程 | 构建期预处理与运行时增量判定

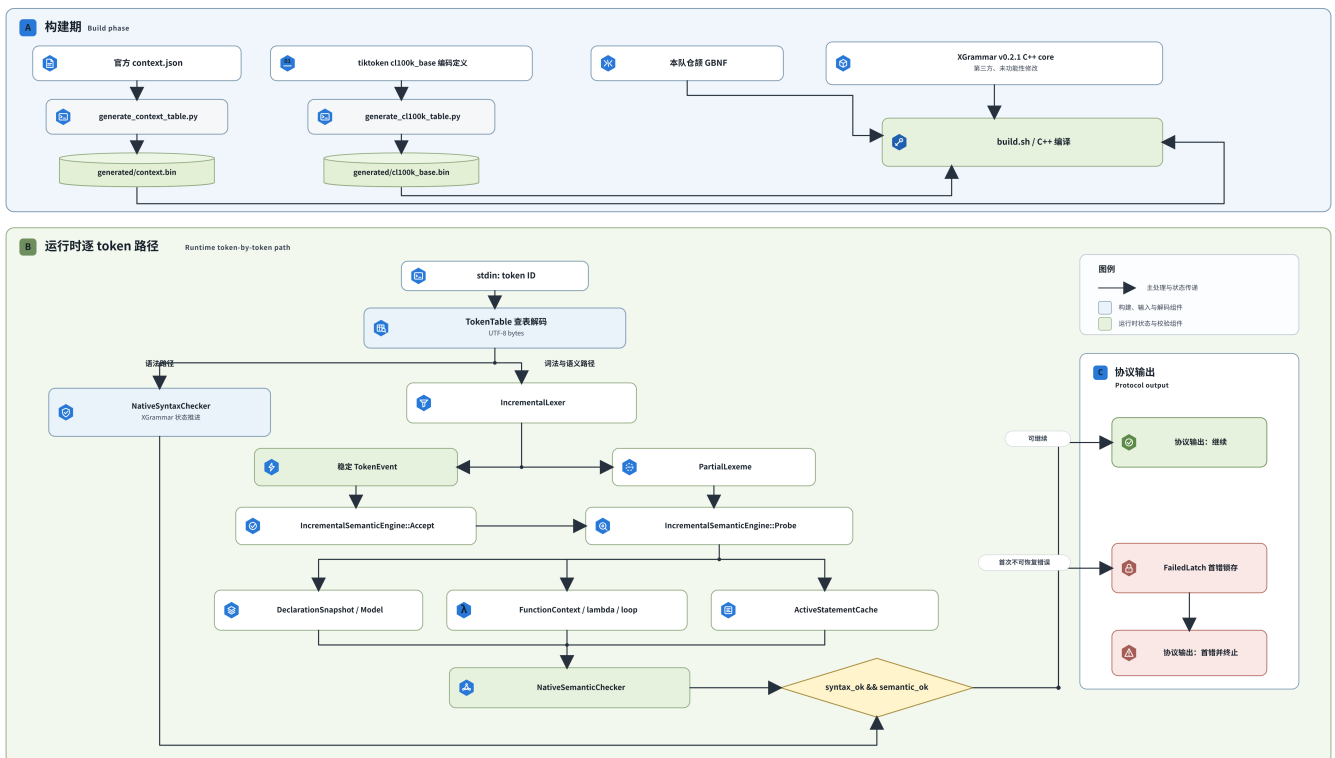
当前加固纯 C++ 版：双层增量检查架构

构建期生成紧凑二进制数据；运行时由 XGrammar 管理字符语法状态，本队 C++ 引擎管理仓颌语义状态。



仓颌增量语法与语义校验系统架构图

BUILD-TIME ARTIFACT GENERATION - RUNTIME TOKEN-BY-TOKEN VALIDATION



tools/generate_context_table.py 只做结构规范化与二进制序列化，生成 generated/context.bin；运行时由 LoadContextTable 将该二进制表作为预定义符号的单一数据源。assets/cl100k_base.bin.xz 在需要时离线解压为 token 查表。提交包直接编译 cpp/solution.cpp、cpp/native_semantic.cpp、cpp/call_frontier.cpp、cpp/continuation.cpp 与随包分发、未作功能性修改的 XGrammar v0.2.1 C++ core。构建不访问网络，生产运行不经过 Python 绑定或 TVM FFI。

token ID

```

→ C++ TokenTable
→ UTF-8 bytes
  ↳ NativeSyntaxChecker
    ↳ 标识符有限商 (`NativeSyntaxChecker::IdentifierGap`)
      ↳ speculative AcceptString / Rollback(1)
        ↳ XGrammar GrammarMatcher → syntax_ok
  ↳ IncrementalLexer
    ↳ stable TokenEvent
    ↳ PartialLexeme
      ↓
      NativeSemanticChecker / AnalyzeSource → baseline
        ↳ complete visible source
        ↳ cached Model / FunctionContext
        ↳ ExpressionTyper
        ↳ ActiveStatementCache
          ↓
          ↳ production metadata:
            ↳ FrontierInfo.line/frontier_end
            ↳ successful closed-call CallFrontier
          ↳ shadow only:
            ↳ PostfixGraph/BFS RecoveryWitness
            ↳ failure-side CallFrontier sample
          ↳ DecisionContext → ComputeProof → DecideWithProof
            ↳ Alive + ValidSuffix → defer
            ↳ Dead + ClosedWorldExhaustive → reject
            ↳ otherwise → baseline verdict
              ↓
              DecisionLedger (trace only)
                ↓
                semantic_ok / failed latch
→ syntax_ok && semantic_ok
→ stdout

```

3.2 主要模块

模块	职责	实现要点
TokenTable	token ID 到 UTF-8 字节片段的还原	构建期生成二进制表，运行时直接查表
NativeSyntaxChecker / XGrammar 语法层	推进仓颉 GBNF 字符级语法状态	调用第三方 XGrammar v0.2.1 C++ core 的通用 GrammarCompiler / GrammarMatcher 能力
Block grammar flattening	限制长 block 的递归状态增长	将语句列表的右递归写法改为等价闭包，不改变可接受语言
标识符有限商	压缩超长普通 ASCII 标识符对语法 matcher 的可观察状态	由 NativeSyntaxChecker::IdentifierGap 与 accept_normalized_and_probe 实现；从文法提取 identifier-shaped literals，按最长窗口保留必要后缀并维护一个可回滚试探
IncrementalLexer	识别稳定 token 与可延长片段	支持任意字节分片、UTF-8、注释、字符串、数值和多字符运算符
NativeSemanticChecker / IncrementalSemanticEngine	接收稳定词法事件并执行 partial/source probe	首错锁存，复用源码、声明快照、活动函数上下文和活动语句
Model / DeclarationSnapshot / FunctionContext	保存预定义符号、声明、作用域和类型关系	context 构建期二进制化；声明快照与结构提交点共同更新活动模型
ExpressionTyper	推导表达式、调用、泛型和 lambda 类型	支持期望类型传播、重载候选与类型变量替换
ActiveStatementCache	维护当前活动语句	增量跟踪括号、字符串、注释和多行结构
FrontierInfo	对失败点的符号、尾部和边界分类	分类 verdict 主要用于 shadow；其中 line/frontier_end 进入开放数组的生产证明上下文
PostfixGraph / RecoveryWitness	生成最多三步、最多 32 状态的有界后缀见证	纯 shadow/诊断，不直接裁决生产结果
CallFrontierClassifier	对每个重载独立维护存活/等待/淘汰状态	失败侧采样主要用于 shadow；源码以) 闭合时重新分类，完整候选集全灭才进入生产判定
ContinuationProof / DecideWithProof	统一包装当前 AnalyzeSource 判定并按证明覆盖	生产覆盖仅允许 ValidSuffix 的 Alive 与 ClosedWorldExhaustive 的 Dead；其余保持当前分析结果
DecisionLedger	记录基线、前沿、证明、候选和规则编号	trace only，不参与裁决
差分与生成工具	验证首错、分片、随机程序和性能	仅用于离线测试与复现

3.3 功能覆盖

决赛第一阶段延续初赛的逐 token 前缀检查要求，同时向测试程序提供额外的全局变量、全局函数、类和接口定义，并在初赛用例之外追加 100 个不公开用例。100 个不公开用例属于最终测试数据规模，不等同于 100 项新语义规则；对实现影响最大的变化，是检查器必须把赛题提供的 context 纳入名称解析、类型推导、调用匹配和名义类型关系。

最终版本已建立完整的 context 数据链路：团队先根据行为审计对来自官方 context 的提交 context.json 做最小适配，再由 tools/generate_context_table.py 结构化序列化为 generated/context.bin，运行时由 LoadContextTable 写入 Model::globals、Model::functions 和 Model::nominals。生成脚本本身不负责 first/last 等行为改写。原先手写的预定义模型被移除，避免同一重载被重复注册或产生官方不存在的成员；--dump-context-ir 可输出规范化运行时模型，供离线工具与来源数据、行为探针结果对比。

3.3.1 决赛新增 context 能力

决赛新增内容	最终版本已实现能力	当前仓库 context 中的定义	主要实现位置
全局变量	接受 <code>global_variables</code> 的名称、类型和可变性字段；构建期写入二进制表，运行时载入全局符号表；表达式中的裸标识符可从 <code>Model::globals</code> 获取类型，并参与未定义名称和类型兼容判断	当前 <code>context.json</code> 的 <code>global_variables</code> 列表为空，因此本文不虚构具体变量名；加载和查找链路已经实现，可处理赛题下发的非空条目	<code>src/context_loader.py</code> 、 <code>tools/generate_context_table.py</code> 、 <code>cpp/native_semantic.cpp::LoadContextTable</code> 、 <code>ExpressionTyper</code>
全局函数	载入函数名、参数名、参数类型、返回类型、泛型参数、必选参数数量和全部重载；支持普通调用、函数值引用、位置/命名/默认参数、重载候选过滤、泛型替换及返回类型检查	8 组函数、23 个重载签名： <code>print</code> 、 <code>println</code> 、 <code>eprint</code> 、 <code>eprintln</code> 、 <code>min</code> 、 <code>max</code> 、 <code>abs</code> 、 <code>clamp</code>	<code>ContextTableReader::Signature</code> 、 <code>Model::functions</code> 、 <code>ExpressionTyper</code> 、调用候选匹配
类定义	载入类名、类型参数、父类型、实例/静态字段、实例/静态方法、构造器及其重载；支持构造器调用、字段访问、静态与实例成员区分、未知成员检查、泛型实参替换和名义子类型判断	11 个类： <code>Array</code> 、 <code>ArrayList</code> 、 <code>HashMap</code> 、 <code>HashSet</code> 、 <code>String</code> 、 <code>Optional</code> 、 <code>KeyValuePair</code> 、 <code>ValuesView</code> 、 <code>Range</code> 、 <code>ArrayStack</code> 、 <code>ArrayDeque</code>	<code>NominalInfo</code> 、 <code>Model::nominals</code> 、 <code>LoadContextTable</code> 、成员访问和构造器检查
接口定义	载入接口名、类型参数、父接口和方法要求；支持接口作为名义类型、泛型接口具体化、沿继承链传递替换、实现方法完整性检查，以及方法泛型参数的 <code>alpha-renaming</code> 签名比较	6 个接口： <code>Hashable</code> 、 <code>Equatable<T></code> 、 <code>Collection<T></code> 、 <code>Iterable<T></code> 、 <code>Stack<T></code> 、 <code>Deque<T></code>	<code>NominalInfo::is_interface</code> 、 <code>CollectInterfaceRequirements</code> 、 <code>SubstituteSignature</code> 、 <code>SameInterfaceSignature</code> 、 <code>CheckInterfaces</code>

结构加载之外，最终版本还区分“JSON 中存在某成员”和“官方 checker 对该成员的实际行为”。HashMap、HashSet 的实例 size/capacity 在字段与方法同名时按字段优先，因此 m.size 可读而 m.size() 不可调用；静态 String.empty 则保留官方允许的读取/调用行为。上述校准来自行为探针和提交消融，不把原始 JSON 结构直接等同于官方前缀语义。

3.3.2 完整赛题功能覆盖

下表中的“支持”限定在赛题仓颌子集、当前 context 和项目测试范围内，不外推到完整仓颌 1.0.0 语言。

赛题能力	支持范围	主要实现	测试证据
基础表达式与运算符	支持赛题整数、浮点、Bool、String、Rune、数组、索引、成员和常见运算符	ExpressionTyper	公开错误样例、单元测试、项目语料
context 全局符号	支持决赛下发的全局变量类型查找、全局函数重载和泛型调用，以及全局函数作为函数值参与期望类型匹配	Model::globals、Model::functions、ExpressionTyper	context API 差分、公开样例

if / while 条件	检查条件是否可兼容 Bool	FunctionContext、ExpressionTyper	err_if_not_bool、err_while_not_bool
for、range 与循环控制	检查 Array/ArrayList/HashMap/Range 等可迭代类型、绑定模式、循环体裸标识符和循环深度	循环上下文、可迭代类型检查	定向样例与随机控制流家族
变量与作用域	支持函数参数、函数体局部变量、同行 let/var、可变性、重复声明、未定义符号、函数间隔离和活动 lambda 参数	FunctionContext、声明快照、活动 lambda 上下文	公开样例、scope-isolation 随机家族、函数隔离回归
函数、方法与构造器调用	支持位置参数、命名参数、默认参数、全局函数、静态/实例方法、构造器和赛题重载规则	Model、FunctionSig、调用候选过滤	公开调用样例、context API 差分、overload 随机家族
类、字段与静态成员	支持 context 类和源码内类的字段、静态字段、实例/静态方法、构造器、泛型类型实参和未知成员检查	NominalInfo、成员访问与构造器检查	类与集合 API 定向回归
接口与继承	支持 context 接口和源码内接口、泛型接口、父接口要求、类实现检查、传递替换和签名规范化	接口要求收集、substitution、alpha-renaming	接口公开样例、generic-inheritance 家族
显式泛型实参	支持数量检查、类型变量替换和具体化签名匹配	substitution、候选匹配	公开样例与随机测试
泛型实参推导	根据实参、期望返回类型和候选约束推导赛题常见形式；歧义时不任意选择候选	ExpressionTyper、类型变量统一	高阶泛型与 min/max 回归
显式参数类型 lambda	支持参数数量、参数/返回类型和高阶函数上下文	活动 lambda 上下文、期望函数类型传播	lambda 固定回归
lambda 参数类型推导	从目标函数类型、重载候选和回调接口传播期望类型；无唯一候选时保留或报告歧义	候选函数类型、活动 lambda 参数	nested-lambda 与高阶调用回归
字段/方法同名行为	实例接收者优先解析字段；静态 String.empty 采用单独行为	字段优先分流、context 行为校准	106 个 context 成员的 A/B/C/D 行为探针矩阵
单步后缀可恢复性	识别一次成员、方法引用、函数调用、索引、静态成员或 Bool 运算能否到达期望类型	MemberRecoversType、OperatorRecoversType	let/var RHS、数组元素定向语料
数组元素提交前沿	基线先做单步正向恢复；逗号处基线重新检查后，若数组整体仍开放，证明层继续以 ValidSuffix("]") 延迟；闭合] 后最终重检	单步恢复、整行方括号平衡、ValidSuffix 证明	1497 项合法前缀语料与后缀验证
调用硬提交前沿	) 到达后，只有完整重载集全部淘汰才判不可续写	CallFrontierClassifier、ClosedWorldExhaustive	调用、默认参数、泛型和命名参数冒烟测试
lambda twin	登记已闭合且通过检查的 canonical lambda body；开放 lambda 的二元尾仅在命中合法 twin 时允许提前提交错误	CanonicalLambdaBody、HasSeenValidLambdaTwin	决赛消融：62~63，且无基线丢分
长 block 状态控制	block 内语句序列采用等价平坦闭包，避免右递归状态随局部声明数快速膨胀	grammar/cangjie.gbnf、grammar/cangjie_token.gbnf	300 个局部声明规模例：>35s → 448~456ms
长标识符状态控制	仅对普通 ASCII identifier gap 启用有限商；raw/string/rune/comment/number 区域完全隔离	NativeSyntaxChecker::IdentifierGap、accept_normalized_and_probe	约 4 KiB 标识符：>30s → 22~33ms；RSS 约 268MiB（未完成）→9.9MiB（完成）

本文的作用域结论覆盖决赛 context 驱动的全局符号、函数重载、类成员、构造器、接口要求，以及初赛已有的函数参数、局部变量、循环绑定、lambda 和泛型场景。当前报告不声称覆盖完整仓颉语言的全部嵌套块遮蔽、复杂闭包捕获和非常规生命周期组合。

4. 核心技术贡献与当前实现伪代码

GBNF、函数与循环上下文、alpha-renaming 和增量缓存属于通用编译技术；XGrammar 的通用 matcher 也属于第三方能力。本队的技术贡献集中在这些技术面向赛题前缀判定的组合与落地：在 BPE token、UTF-8 字节和仓颉词法 token 三种边界之间建立稳定词法事件/未完成词素分流；将完整程序语义规则改造成保守的不可恢复错误判定；让 XGrammar 语法状态与原生 C++ 语义探测协同推进；消除每 token 全量解析；将 context 和 cl100k 词表移到构建期二进制化，并针对最终版本的 context API 差分补齐语义规则。

当前版本中的 TokenEvent 主要承担词法稳定性划分。语义层的增量性来自缓存模型、活动上下文、活动语句和按提交点触发的选择性重建，所有语义事实尚未转换为由 TokenEvent 执行的单次提交。

4.1 稳定词法事实与可延长词素分流

BPE token、UTF-8 字节和仓颉词法 token 具有不同边界。加固版使用 TokenEvent 表示已经稳定的词法边界，使用 PartialLexeme 保存仍可延长的末尾片段。语义层结合完整可见源码、缓存模型、活动上下文和活动语句执行选择性探测。

算法 1：增量前缀检查主流程

输入：本轮收到的 UTF-8 字节片段 fragment
状态：source、IncrementalLexer、SemanticEngine、failed

CHECK_FRAGMENT(fragment):
 if failed:
 return ERROR

 source ← source + fragment
 (stable_events, partial) ← lexer.FEED(fragment)

 for event in stable_events:
 semantic_engine.ACCEPT(event)

 status ← semantic_engine.PROBE(partial, source)
 if status is ERROR:
 failed ← true

 return status

PROBE 根据本轮新增字符决定活动模型和上下文的更新范围：

PROBE(partial, source):
 delta ← source 中本轮新增的部分

 if delta 包含类、接口、函数或结构提交标记:
 active_model ← preload_model
 收集新增 import、函数、类和接口

 if delta 包含作用域或语句边界:
 更新当前函数、局部变量、循环和 lambda 上下文

 active_statement ← statement_cache.UPDATE(source)

```

return ANALYZE_SOURCE(
    source,
    active_model,
    model_dirty,
    commit_dirty,
    active_context,
    active_statement
)

```

对应实现为 `IncrementalLexer::Feed`、`NativeSemanticChecker::Check`、`IncrementalSemanticEngine::Accept`、`IncrementalSemanticEngine::Probe` 和 `ActiveStatementCache::Update`。`Accept` 当前保存稳定词法事件，主要语义判断由 `Probe` 完成。`Probe` 使用 `delta` 判断模型和上下文是否需要更新，并对当前活动语句执行类型、调用和作用域检查。因此，本文所称“增量语义”指停用每轮 Lark/完整 `typechecker` 深检、复用声明模型与活动上下文，并按提交点选择性更新；当前实现尚未把全部语义规则转换成纯事件驱动状态机。

4.2 前缀可续写性判定模型

4.2.1 定义与状态

从赛题定义看，给定当前源码前缀 p ，若存在某个后缀 s ，使 $p + s$ 成为赛题仓颉子集中语法、类型和作用域均合法的完整程序，则称 p 可续写：

$$\text{CONTINUABLE}(p) \Leftrightarrow \exists s, \text{VALID_PROGRAM}(p + s)$$

最终版本只在少数正式规则中携带最小可打印后缀或候选穷尽证明，不枚举一般表达式的所有补全，也不对完整仓颉语言给出形式化可达性证明。系统采用面向赛题子集的保守错误判定方法：未完成结构优先延迟判断；仅在语法状态无后继，或某条已实现语义规则能够确认后后续输入无法修复时锁存错误。持久状态和单轮工作集可概括为：

```

PersistentPrefixState =
    LexerState
    × GrammarState
    × GlobalModel
    × FunctionContext
    × ActiveLoopContext
    × ActiveLambdaContext
    × ActiveStatement
    × FailedLatch

ProbWorkingSet =
    PartialLexeme
    × CandidateTypes
    × CandidateCalls
    × ExpectedLambdaType

```

`CandidateTypes`、`CandidateCalls` 和 `lambda` 期望类型在当前活动表达式的 `probe` 中构造；结构提交点到达后，相关声明模型和活动上下文按需更新。概念上，单轮判定分为三类：

内部结果	含义	对外判断
NO_ERROR_DETECTED	当前稳定事实没有触发已实现的不可恢复错误规则	可继续
DEFERRED_INCOMPLETE	当前词素、表达式、调用或 <code>lambda</code> 尚未闭合，语义判断被延迟	可继续
IRRECOVERABLE_ERROR	已满足某条后续输入无法修复的错误条件	错误并锁存

当前 C++ 接口使用 `CheckStatus.ok` 对外返回，因此 `NO_ERROR_DETECTED` 和 `DEFERRED_INCOMPLETE` 均对应 `ok=true`，`IRRECOVERABLE_ERROR` 对应 `ok=false`。返回“可继续”表示当前没有触发已实现的不可恢复错误规则，或结构尚未闭合而需要延迟判断。最终平台评分、context 行为审计、合法前缀扫描和定向回归分层验证该工程判定；第 7 节同时如实记录最终扫描仍存在的 38 次 early fire。

4.2.2 不可恢复错误判定

语义检查仅在某条已实现规则确认当前错误不受未来后缀影响时锁存错误；无法确定时采用延迟判断。主要判定条件如下：

错误类别	最早判定条件	后续输入无法修复的原因
语法自动机无后继	XGrammar 拒绝本轮新增字节，当前语法状态不存在对应转移	已输入字符无法删除或改写
break / continue 越界	关键字已经稳定，当前函数的循环深度为 0	后续新增循环无法包围已经出现的控制语句
let 重新赋值	左值和赋值语句达到提交点，符号已确定为不可变	后缀无法改变已有声明的可变性
未定义标识符	标识符词素与所在表达式已稳定，且不属于可见符号、类型、关键字或其可延长前缀	后缀无法修改已经提交的名称文本和既有可见域
条件类型错误	if / while 条件闭合，推导类型与 Bool 不兼容	后缀无法重新打开已闭合条件表达式
调用无候选	参数列表闭合，全部重载因名称、数量、命名参数或类型不匹配被排除	后缀无法向已闭合调用补入参数或恢复候选
数组元素类型错误	目标元素类型已知，包含该元素的数组已由] 闭合，当前元素仍与目标类型不兼容	数组闭合后不能再向其其中的元素追加成员、调用、索引或运算后缀
返回类型错误	return 表达式达到语义提交点，推导类型与函数结果类型不兼容	后续语句无法修改已提交的返回表达式
重复声明	声明名称、声明种类和所在作用域已经确定	后缀无法撤销同一作用域中的既有声明

以 `let a: Array<Int64> = ["x"` 为例，当前元素暂时推导为 `String`，但数组仍开放，未来字符仍可能构成成员、调用或其他已建模后缀，因此最终版本不在这一位置用负搜索提前判死。到 `]` 提交后，表达式被重新检查；若完整元素仍不能兼容 `Int64`，才在该硬边界形成不可恢复错误。不同 `cl100k`/字节分片可能改变边界在哪一轮凑齐，官方首错锚点用于约束平台协议下的拒绝轮次。默认公开 checker 协议在拒绝轮输出 1，`--competition-output` 模式在同一轮输出 0。

4.2.3 PartialLexeme 处理原则

- 未完成标识符：当文本仍是可见符号、类型名或关键字的前缀时延迟判断；分隔符到达后再提交完整名称。
- 未完成数值：保留整数、浮点数、后缀和指数形式等仍可形成的候选；数值词素稳定后提交具体类型。
- 未完成字符串：保留字符串闭合和转义状态，同时允许表达式层使用已经确定的 `String` 类型类别做保守推导。
- 未完成运算符：`<`、`>`、`=`、`.` 等可能继续形成多字符运算符，在最长可行形式稳定前保留语法分支。
- UTF-8 中间字节：完整码点形成前只推进字节/词法状态，不提交语义名称或字面量。
- 未闭合调用或 `lambda`：保留仍可满足的重载、参数类型和期望返回类型；闭合前避免任意选定单一候选。

4.2.4 设计不变量

1. 已提交的 `TokenEvent` 不会被未来输入改变词素边界或类别；
2. 未闭合且当前无法确定错误不可恢复的结构采用延迟判断；
3. 仅在某条已实现规则确认当前错误不受未来后缀影响时锁存错误；
4. 错误锁存后，后续输入不会恢复为可续写状态；

5. 分片布局只影响事件到达轮次，拼接后的相同字符流应得到一致的最终结论。

4.2.5 复杂度与最坏情况

令 Δ 为本轮新增字节数， n 为当前累计源码长度， a 为活动语句长度， c 为调用/重载候选数。

环节	常见开销	最坏情况与触发条件
token ID 解码	查表后复制本轮字节，约 $O(\Delta)$	与解码片段长度线性相关
XGrammar 状态推进	与新增字节数及语法状态转移代价相关	复杂语法状态下转移代价增加
IncrementalLexer::Feed	扫描新增字节和当前 pending 尾部，约 $O(\Delta + \text{pending})$	超长未闭合字符串/注释会重复扫描增长中的 pending，累计可能达到二次量级
ActiveStatementCache::Update	函数体稳定时从 cursor 扫描新增字节，摊销接近 $O(\Delta)$	函数上下文切换或缓存重置后需要重新建立活动状态
表达式与调用分析	约 $O(a \times c)$ ，具体取决于嵌套表达式和候选过滤	超长活动表达式、大量重载或嵌套泛型会放大开销
模型与上下文更新	大多数普通字节复用旧模型	类、接口、函数或结构提交点可能触发对当前源码的局部模型重建或较大范围扫描，单轮可达 $O(n)$
当前 Probe 边界扫描	每轮仍需计算部分结构状态	当前实现中部分花括号/上下文探测会扫描可见源码，单轮最坏为 $O(n)$ ；后续可用持久化结构帧继续优化

因此最终实现的平均路径具有明显增量收益，最坏路径仍保留全前缀或增长 pending 的重复工作。历史旧加固纯 C++ 版本的 8.55×（历史旧加固版）数据只反映初赛旧平台公开负载，不用于证明最终版本的复杂度或直接性能。

4.3 活动语句缓存覆盖多行表达式

ActiveStatementCache 从上次 cursor 继续扫描本轮新增字符，维护圆括号、方括号、花括号、字符串、转义、行注释和嵌套块注释状态。语句提交仅发生在顶层分号或满足条件的顶层换行位置。

算法 2：活动语句增量维护

```
UPDATE(source):
    从上次 cursor 继续扫描新增字符

    遇到字符串或注释：
        更新字符串、转义和注释状态

    遇到左括号、方括号或花括号：
        对应深度加一

    遇到匹配的右括号：
        对应深度减一

    if 当前位于顶层 and 遇到分号或可提交换行：
        提交上一条完整语句
        更新 statement_start

    return source[statement_start : 当前可见末尾]
```

该缓存覆盖多行变量声明、多行调用、嵌套 lambda，以及字符串和注释中的括号。语义层由此获得结构完整的活动语句视图。

4.4 传递泛型接口要求与 *alpha-renaming*

加固版沿接口继承链替换类型实参，使用 `visited` 集合控制重复与循环访问；方法签名比较前，将接口方法的泛型参数映射到实现方法的泛型参数，实现 *alpha-renaming*。

算法 3：泛型接口实现检查

```
COLLECT_REQUIREMENTS(interface_type, visited):
    interface_type ← 规范化(interface_type)

    if interface_type in visited:
        return
    visited.add(interface_type)

    interface ← 查找接口定义
    substitution ← 接口类型参数到实际类型实参的映射

    for required_method in interface.methods:
        requirements.add(
            SUBSTITUTE_SIGNATURE(required_method, substitution)
        )

    for parent_interface in interface.supers:
        concrete_parent ← SUBSTITUTE(parent_interface, substitution)
        COLLECT_REQUIREMENTS(concrete_parent, visited)

SAME_SIGNATURE(implementation, requirement):
    if 泛型参数数量或方法参数数量不同:
        return false

    rename ← requirement 方法泛型参数
        到 implementation 泛型参数的逐项映射
    normalized_requirement ← SUBSTITUTE_SIGNATURE(requirement, rename)

    return 参数类型逐项相同
        and 返回类型相同
```

对应实现为 `CollectInterfaceRequirements`、`SubstituteSignature` 和 `SameInterfaceSignature`。该路径支持 `Interface<T>` 的传递继承、具体化接口要求和方法泛型参数规范化。

4.5 决赛 *context* 语义加固

最终版本在原有行声明、嵌套 lambda、泛型接口继承、重载消歧和作用域隔离基础上，进一步加固以下语义规则：

决赛加固项	当前处理方式
同行 var/let 声明	在活动语句中维护尚未到达换行或分号提交点的声明状态；声明名称和类型一旦形成，就先加入当前可见上下文，使同一行后续表达式能够正确引用。
循环体裸标识符	识别不带花括号的单语句循环体，将循环绑定变量和当前作用域传入该语句的表达式检查，避免只支持块状循环体。
字符串和注释伪声明屏蔽	在扫描声明、字段和构造器之前，将字符串、行注释和嵌套块注释替换为等长空白区域；保持源码位置不变，同时阻止其中的关键字被识别为真实代码。
var 字段构造器初始化	收集类中的可变字段和各构造器赋值情况，在构造器闭合时检查必须初始化的字段是否已经赋值，并区分局部变量、参数遮蔽和真实字段写入。
尾随空参数	调用参数列表闭合时，只提交实际存在的最后一个参数；若逗号后直接遇到右括号，则不再生成空参数，避免参数数量被重复计算。
min/max 泛型行为	对显式和隐式泛型调用分别处理：显式类型实参检查数量和替换结果，隐式调用根据实参及期望返回类型推导；在调用尚未闭合时保留候选，闭合后再确定参数或泛型错误。
Array 首末属性	first/last 按 Optional<T> 实例字段/自动应用属性进入模型，不再错误建模为零参数方法。
字段/方法同名冲突	HashMap、HashSet 实例上的 size/capacity 按字段优先；静态 String.empty 保留官方允许的读取/调用行为。
成员前缀提交	已知接收者的 .member 若不是任何真实成员名的前缀，则在成员 token 已稳定时提交未知成员错误。
数组与调用边界	数组基线先做单步恢复；仍产生的元素拒绝若处于开放数组，则由 ValidSuffix("]") 延迟，闭合] 再由原基线重新判定；闭合调用仅在全部重载候选被完整淘汰时判死。

这些加固总体上优先保护未完成结构的可续写性：只有名称、元素、调用、构造器等对象达到相应规则定义的提交边界，且错误不可由后续输入修复时才报告。这里的“提交边界”不等于一律闭合；成员前缀、单步 RHS 等定向规则仍可能在稳定的开放值 token 处提交。证明覆盖层只在持有正式证据时改变当前 AnalyzeSource 结果，且最终前缀审计仍如实记录 38 次 early fire。

4.6 证明携带的续延判定与决策台账

最终版本将原有语义判定作为 baseline，由统一的 DecideWithProof 包装。每个可覆盖决定都必须携带 ContinuationProof；无证明、证明类型不被允许或前沿仍为 Unknown 时，结果保持基线。DecisionLedger 记录决策位置、基线结果、前沿状态、证明种类、类型信息、候选数、规则编号和可打印后缀，便于离线审计。

算法 4：证明携带的前缀判定

```
DECIDE_PREFIX(baseline, message, source, frontier, witness, call_frontier):
    context ← MAKE_DECISION_CONTEXT(
        message, source, frontier, witness, call_frontier
    )
    proof ← COMPUTE_PROOF(context)
    LEDGER.APPEND(context, baseline, proof)

    if proof.state = Alive and proof.kind = ValidSuffix:
        return CONTINUABLE

    if proof.state = Dead and
       proof.kind in {OfficialAudit, ClosedWorldExhaustive}:
        return ERROR
```

```

return baseline

COMPUTE_PROOF(context):
    if context.site = ARRAY_ELEMENT and context.element_open:
        return Proof(
            state = Alive,
            kind = ValidSuffix,
            rule = "array-element-open",
            printable_suffix = "]"
        )

    if context.site = CALL_CLOSE and context.call_closed and
        context.candidate_count > 0 and context.alive_count = 0:
        return Proof(
            state = Dead,
            kind = ClosedWorldExhaustive,
            eliminated = context.elimination_reasons
        )

    return Proof(state = Unknown, kind = None)

```

当前生产实现只有两条正式覆盖规则：开放数组元素的 Alive + ValidSuffix，以及闭合调用候选穷尽的 Dead + ClosedWorldExhaustive。数组规则记录的 "]" 只是把检查推进到既有基线重新判定处的最小可打印提交标记，不是能单独把任意前缀补成合法程序的完整后缀；运行时既不搜索完整 program tail，也不调用官方 checker。237/237 的正向证据来自离线把原程序剩余 tail 接回后复验。OfficialAudit 已作为证明类型保留，但当前没有以它激活新的生产规则；PostfixGraph 的 BFS RecoveryWitness 仍主要用于 shadow 观测，不能与生产判定混称。

4.7 逐重载调用前沿与硬提交边界

CallFrontierClassifier 为每个重载维护独立状态。已经提交的实参数量超过形参数量、已知类型与非符号形参不兼容，或调用闭合后仍缺少必选参数，才淘汰候选；未知实参类型和仍含未绑定类型变量的形参不会被用作负证明。只有右括号已经提交且存活候选为零时，调用前沿才形成封闭世界的 Dead。

算法 5：逐重载调用前沿

```

CLASSIFY_CALL_FRONTIER(overloads, typed_args, call_closed, expected_type):
    alive_count ← 0
    reasons ← []

    for overload in overloads:
        state ← Alive

        for (index, arg_type) in typed_args:
            if index >= overload.parameter_count:
                state ← Dead(ARITY_EXCEEDED)
                break

            parameter_type ← overload.parameters[index]
            if arg_type is Unknown or parameter_type has unbound type variable:
                continue

            if not COMPATIBLE(arg_type, parameter_type):
                state ← Dead(ARG_TYPE_MISMATCH)
                break

        if state is not Dead and call_closed and

```

```

typed_args.count < overload.required_count:
    state ← Dead(ARITY_SHORT_AT_CLOSE)

if state is not Dead:
    alive_count ← alive_count + 1

reasons.APPEND(overload, state)

return (alive_count, reasons, call_closed)

```

期望返回类型只用于记录和后续上下文判断，不会单独淘汰重载；这样可以避免在调用仍开放时，把可能通过后缀或外层期望恢复的表达式过早判死。

4.8 单步可恢复性与 *lambda twin*

决赛消融表明，宽 BFS 恢复会跨越过多尚未建模的行为边界。最终版本在生产 let/var RHS 等普通开放值位置采用更窄的“一次后缀”可恢复性：一次字段/静态字段、方法结果、方法引用、函数值调用、数组/列表/String 索引，或产生 Bool 的一次运算能够达到期望类型时，开放表达式才保留延长空间；两步链不计入该生产规则。数组元素另由 4.6 节的开放数组证明统一延迟到 1，不以单步搜索失败作为 Dead。

算法 6：单步可恢复性与提交

```

ONE_POSTFIX_RECOVERABLE(actual_type, expected_type, model):
    if actual_type is Function(P...) -> R and COMPATIBLE(R, expected_type):
        return true

    if actual_type is Array<T> or ArrayList<T> and COMPATIBLE(T, expected_type):
        return true
    if actual_type is String and expected_type = Rune:
        return true

    for field or method in INSTANCE_AND_STATIC_MEMBERS(actual_type):
        concrete_result ← SUBSTITUTE_RECEIVER_TYPE_ARGUMENTS(member.result)
        if COMPATIBLE(concrete_result, expected_type):
            return true
        if expected_type is function type and
            COMPATIBLE(METHOD_VALUE_TYPE(member), expected_type):
            return true

    if expected_type = Bool and actual_type is known non-function type:
        return true

    return false

CHECK_OPEN_RHS(actual, expected, boundary):
    if boundary is open and ONE_POSTFIX_RECOVERABLE(actual, expected, model):
        return DEFERRED_INCOMPLETE
    return CHECK_COMPATIBILITY_AT_COMMIT(actual, expected)

```

lambda 另有一条经平台消融保留的窄机制。系统只登记已经闭合且通过类型检查的 lambda body，并去除空白形成 canonical 文本；开放 lambda 出现二元运算尾部时，只有当前 body 前缀命中此前合法 twin，才允许按该已验证形态提前提交错误，否则一直延迟到 }。

算法 7：lambda twin 锚定

```

ON_LAMBDA_CLOSED(lambda_body, typecheck_ok):
    if typecheck_ok:
        VALID_LAMBDA_BODIES.ADD(CANONICALIZE(lambda_body))

CHECK_OPEN_LAMBDA(body_so_far, inferred_error):
    if not inferred_error:
        return CONTINUABLE

    if TAIL_IS_BINARY(body_so_far) and
        PREFIX_MATCHES_VALID_TWIN(CANONICALIZE(body_so_far)):
        return ERROR

return DEFERRED_INCOMPLETE    // 等待 `}` 硬边界

```

4.9 文法平坦化与标识符有限商

长 block 的语句表原本采用右递归，随着局部声明增多会积累大量等价语法状态。最终版本把它改写为等价的零次或多次闭包；语言不变，但 matcher 不再为每条语句保留一层递归尾部。

算法 8: block 语句表平坦化

旧形式:

```
BLOCK_ITEMS := STATEMENT WS BLOCK_ITEMS | STATEMENT
```

最终形式:

```
BLOCK_ITEMS := STATEMENT (WS STATEMENT)*
```

约束:

STATEMENT 的定义和分隔规则保持不变
对旧、新文法执行同一合法/非法语料差分

另一类退化来自超长普通标识符：仓颉文法只能通过关键字和少量 identifier-shaped 字面量观察其内容，但通用 matcher 仍可能为每个字符保留大量等价状态。最终版本在语法层之前自动提取这组可观察字面量，选择一个完全不出现在任何 identifier-shaped literal 中的 canonical 字符，压缩已经脱离所有可观察前缀的 gap。系统只保留一步 speculative AcceptString：下一片段相同或属于安全延长时复用该 probe；若新的 normalized 输出不再提交同一 probe，则先 Rollback(1)，再提交重新计算的 normalized 代表，而不是回退到原始逐字节路径。

算法 9: 文法可观察的标识符有限商

```
BUILD_OBSERVABLE_LITERALS(grammar):
```

```

    literals ← 提取形如普通标识符的终结字面量
    window ← max(length(literal) for literal in literals)
    canonical ← 选择完全不出现在任何 literal 中的 ASCII 标识符字符
    return (literals, window, canonical)

```

```
FEED_SYNTAX_FRAGMENT(fragment, lexical_mode, gap_state):
```

```

    if lexical_mode not in PLAIN_ASCII_IDENTIFIER:
        gap_state.RESET()
        return MATCHER.ACCEPT(fragment)

```

```

    (stable_text, optional_probe) ←
        NORMALIZE_IDENTIFIER_GAPS(fragment, literals, window, canonical)

```

```

    if an earlier speculative probe is active:
        if current probe is identical or is a safe identifier-gap extension:

```

```

        reuse the live probe and return ACCEPT
    else if stable_text commits the same probe:
        remove that committed prefix from stable_text
    else:
        MATCHER.ROLLBACK(1)
        clear earlier probe state

    if stable_text is not empty and not MATCHER.ACCEPT(stable_text):
        return REJECT

    if optional_probe exists:
        if not MATCHER.ACCEPT(optional_probe):
            return REJECT
        remember it as the one live speculative step

    return ACCEPT

```

有限商严格隔离 raw identifier、字符串、Rune、注释和数字，只对词法器确认的普通 ASCII identifier gap 生效。它优化的是“文法对内容不可区分”的状态，不改变语义层看到的完整源码与原始标识符文本。

5. 重难点、修改历程与解决方案

5.1 BPE 分片与仓颉词法边界

- 现象：标识符、UTF-8 字符、数字、多字符运算符、字符串和嵌套注释可能跨越多轮输入。
- 根因：LLM token 边界无法直接作为仓颉 lexer 的提交边界。
- 解决方法：C++ IncrementalLexer 维护 pending buffer，输出稳定 TokenEvent，将末尾可延长内容保留为 PartialLexeme。
- 验证证据：byte、random、line、cl100k 和 whole 五种分片得到一致结果。

最终版本补充：同一活动语句中的 var/let 待提交状态、字符串/注释掩码和尾随空参数修复，都建立在稳定词法事实与源码结构边界分离之上；相关生产实现集中在 cpp/native_semantic.cpp，并由决赛阶段的定向回归覆盖。

5.2 合法未完成前缀的提前误报风险

- 现象：标识符、字符串、函数参数或 lambda 的中间状态仍有合法补全路径。
- 根因：完整程序 typechecker 的诊断对象是完整语法结构，前缀检查还需判断未来延长空间。
- 解决方法：稳定词法边界保存、partial probe、可见符号前缀判断和首次错误锁存协同工作；最终版本再在证明/BFS 层引入 Alive/Dead/Unknown 三态和硬提交边界，该层搜索不到恢复后缀时只得到 Unknown。普通 AnalyzeSource 基线仍保留成员前缀、单步 RHS 等定向提交规则。
- 验证证据：初赛历史语料中 180 个合法程序在全部观测前缀上零误报。决赛的 1497 项官方接受程序前缀审计则用于发现最终机制的剩余早拒问题，结论单独见 5.10；两组证据不混为同一版本的“零误报”。

5.3 每 token 重复扫描与深度解析

- 现象：初始基线的 50 例平均耗时为 2.33746s。
- 根因：每轮输入都会重新扫描完整源码前缀，并执行 Lark 解析和类型检查。历史性能分析记录了公开案例约 5575 次深检，

Lark/typechecker 约占原热路径 83%。

- 解决历程：纯 Python 版缓存声明与上下文；混合版迁移协议、解码和 XGrammar 状态；纯 C++ 版迁移语义热路径；构建期生成 cl100k 和 context 二进制表；决赛再对长 block 的右递归状态和长普通标识符的等价语法状态做窄化处理。
- 验证证据：历史旧加固版平均耗时 0.27328s，相对基线提速 8.55×（历史旧加固版）；该数据不代表最终版本。

5.4 多行函数签名进入语义模型

- 现象：参数列表跨行时，旧函数和方法模式可能漏收集声明。
- 根因：早期快速匹配路径限定参数位于单行。
- 解决方法：保留单行快速路径；检测到多行函数头时启用宽模式；函数收集、当前函数选择和声明类型检查使用统一分流条件。
- 验证证据：多行泛型函数、多行方法和多行调用进入固定回归及随机生成家族。

最终版本补充：多行声明之外，最终版本还处理同行声明上下文与循环体裸标识符，使物理换行和花括号形式不再成为语义可见性的唯一依据；由 context API 差分回归覆盖。

5.5 嵌套 lambda 参数可见性

- 现象：2-4 层嵌套 lambda 和高阶函数调用可能将合法参数识别为未定义名称。
- 根因：未闭合 lambda 的参数尚未进入普通函数作用域。
- 解决方法：扫描活动花括号栈，识别 { parameters => body }，将活动 lambda 参数加入表达式上下文，并传播期望函数类型。
- 验证证据：显式 lambda、推导 lambda、高阶函数、类静态方法和接口回调案例进入回归。

5.6 泛型接口传递继承与签名规范化

- 现象：接口继承链上的类型实参替换可能缺失；方法泛型参数名称差异可能形成虚假不匹配。
- 根因：接口要求需要沿继承链具体化，方法级类型参数还需要独立规范化。
- 解决方法：递归收集接口要求、沿链应用 substitution、使用 visited 控制循环、比较前执行 alpha-renaming。
- 验证证据：传递泛型接口继承、具体化实现和方法泛型签名进入随机回归。

5.7 重载方法引用歧义

- 现象：多个裸成员方法候选同时适用时，单个候选选择会掩盖歧义。
- 根因：方法引用缺少调用实参，普通调用匹配无法完成消歧。
- 解决方法：多个适用候选返回歧义；唯一候选构造函数类型；接收者泛型实参先完成替换。
- 验证证据：唯一方法引用和模糊方法引用分别进入生成器。

5.8 函数作用域与关键字边界

- 现象：函数参数可能进入后续函数上下文；`foreign...` 可能触发 `for` 语句路径。
- 根因：切换函数时状态清理范围不足，关键字检测只判断字符串前缀。
- 解决方法：切换函数时清空前一函数参数和不可变集合；`StartsWithKeyword` 同时检查关键字文本和后续标识符边界。
- 验证证据：跨函数作用域隔离和关键字前缀标识符纳入随机测试。

最终版本补充：作用域隔离继续作为最终版本回归基线；同时增加非代码文本掩码，避免字符串和注释中的关键字、字段或构造器文本污染真实声明扫描。

5.9 context 结构与官方行为不一致

- 现象：原始 JSON 同时出现字段与方法，或把属性以近似方法的形态列出时，机械加载结构并不能复现官方 checker 的调用行为。
- 根因：context 描述的是可用 API 的结构数据，而成员解析优先级、自动应用属性等仍包含语言行为语义。
- 解决历程：决赛先以官方 context 为来源完成定向行为校准，再移除手写 `AddBuiltinModel`，建立“经校准的提交 `context.json` → `generated/context.bin` → `LoadContextTable`”单一数据链；随后对 106 个 context 成员分别生成 A/B/C/D 探针矩阵，覆盖值读取、调用、方法引用和后缀四类形态。
- 保留修复：`Array.first/last` 按 `Optional<T>` 实例字段/自动应用属性处理，使平台由 60~62；`HashMap/HashSet` 的 `size/capacity` 在实例接收者上字段优先；静态 `String.empty` 使用单独的官方行为分流。
- 边界：行为审计用于发现和校准差异，不意味着这 106 个成员的探针矩阵所代表的全部潜在差异已经完全消除。

5.10 开放表达式、合法续写与硬提交边界

- 现象：在开放数组、调用或 `lambda` 中，当前类型不匹配可能被一次成员、调用、索引或运算后缀修复；反过来，闭合调用在全部重载候选淘汰后又应立即提交。
- 失败原因：把“当前搜索不到后缀”当成 `Dead` 会提前拒绝合法程序；只做无条件延迟又会错过精确首错。
- 解决方法：数组基线先使用单步正向恢复减少早拒；若仍在开放数组内部产生元素拒绝，证明层以 `ValidSuffix` 记录最小提交标记 `"l"` 并统一延迟。该标记不是完整合法程序后缀，运行时不搜索 `program tail`，也不调用官方 checker。调用到) 后由 `CallFrontier` 对完整重载集做封闭世界淘汰；数组的开放/闭合状态使用整行方括号平衡，而不是只观察局部尾串。
- 语料校准：一次决赛审计扫描 1497 个官方接受程序，阶段性报告了 244 次早拒，其中 237 次属于数组元素。数组 `Alive` 覆盖将这 237 项全部延迟；离线工具把各自原程序的完整剩余 `tail` 接回后，由官方检查器复验为 237/237 接受。首次实现曾把闭合 `[1, "x"]` 误当开放，随后以整行括号平衡修正。
- 剩余边界：最终计分源码的复扫仍记录 38 次 `early fire`，其中 31 次为 `postfix initializer`，另有 `method reference`、`clamp`、`array-index` 和 `generic` 等类别；因此本文不声称最终版本已实现所有合法前缀零早拒。

5.11 宽恢复策略的回退与证明化收敛

- 现象：启用较宽恢复候选（其中包含生产 `LetRhsRecoverable`）的版本在官方评测中从 63 降到 59：新增 1 项的同时丢失 5 项既有 F1 家族用例。
- 归因边界：该候选相对对照的唯一生产行为变化使 `LetRhsRecoverable` 成为主要嫌疑，但没有完成单因素官方重跑，因此具体因果仍是推断，不能写成已证实的 BFS 根因。可以确定的设计原则是：有界搜索找不到路径只说明当前抽象不完备，即 `Unknown`，不能单独构成开放表达式不可续写的负证明。

- 回退与收敛：否决该宽恢复候选升入正式版本，将 PostfixGraph/BFS RecoveryWitness 保留为 shadow；生产路径改为窄的一步正向恢复、开放数组的 ValidSuffix 和闭合调用的 ClosedWorldExhaustive。无正式证明时，证明层保持当前 AnalyzeSource 结果；该结果已包含第六补丁，并不等同于字节冻结的早期 63 分源码。
- 结果：最终证明前沿没有在本次隐藏集上带来净新增通过项，但保持了 63.00 分，并把未来新增规则约束为“能说明为什么覆盖基线”的可审计形式。
- 交付修复：打包阶段还发现过运行时 grammar/src 缺失与构建硬失败；最终提交保留压缩 token 表、可重生成且随包分发的 context 二进制表，以及 vendored XGrammar core，实现离线自包含构建。

5.12 长前缀规模退化与窄优化

- 现象：短例耗时正常并不能排除状态规模悬崖。右递归 block 在 300 个局部声明时超过 35s；约 4 KiB 的普通标识符在旧语法路径上超过 30s 仍未完成，峰值 RSS 约 268MiB。
- 否决方案：通用 ranged-state 尝试可把 4 KiB 标识符降到约 0.43s，却令 300 locals 恶化到约 5.38s，其他规模样例最坏回退 7.397×，因此整体回滚。
- 最终方案：G1 将右递归语句表改为等价平坦闭包；G4 只对词法器确认的普通 ASCII identifier gap 构造文法可观察有限商，并用一步可回滚试探与原 matcher 对齐。raw identifier、字符串、Rune、注释和数字保持原始路径。
- 结果解释：G1 将 300 locals 降到约 448–456ms；G4 将 4 KiB 标识符降到 22–33ms，完整运行 RSS 约 9.9MiB。它们对官方短例 SUM 的改善分别约 3.896% 和 0.706%，因此贡献是消除规模悬崖，而不是宣称所有短例普遍大幅提速。

6. 历史性能测试与最终平台结果

6.1 实验环境与统计方法

本节包含两套不可直接混算的数据。决赛数据来自平台结果页，主指标是页面列出的 63 个条目的算术平均；由于页面没有列出其余用例的耗时，本文不把该平均值称为 100 个隐藏用例的总体平均。历史数据来自同一批 50 个公开错误案例，环境为 0522_cangjie_fragment_checker 镜像、Ubuntu 22.04、ARM；历史提速按“简单适配基线平均耗时 ÷ 对比版本平均耗时”计算。

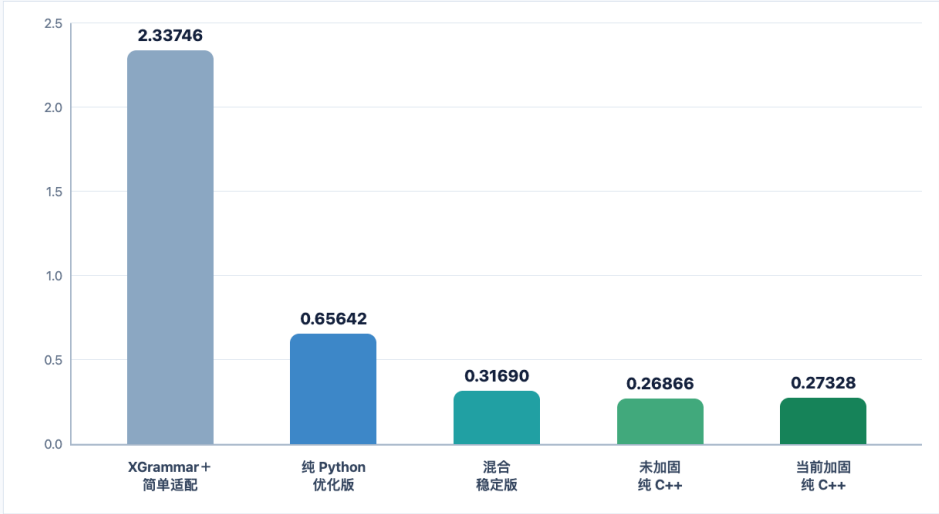
历史冷进程 p50/p95 只用于三个原生化阶段的辅助分析。决赛得分、决赛列出条目的耗时和初赛历史提速分别报告，避免跨样本推导倍数。

6.2 决赛最终提交平台结果

性能演进 | 同一批 50 个公开案例的平台平均耗时

五阶段优化：最终加固版相对基线提速 8.55x

所有数据来自赛题限定环境；柱高为平台平均耗时，单位为秒。



8.55x

最终相对基线提速

平均耗时从 2.33746s 降至 0.27328s。

88.31%

平均耗时降低

50 个案例全部取得正向加速。

+1.72%

加固覆盖成本

相对未加固版，增加多行、lambda、接口、重载与作用域检查。

n=50 | 0522_cangjie_fragment_checker
Ubuntu 22.04 | ARM | AC

指标	最终结果
得分 / 判定	63.00 / WA
平台列出的计时条目	63
累计耗时	16.982s
算术平均耗时	0.2695556s，报告取 0.26956s/条
中位数	0.270s
p95（线性插值）	0.2977s
最小值	0.242s（err_string_compare_indexof）
最大值	0.315s（err_deque_add_last_drift）

平台没有提供正式题型分类，本文不按案例名自行推导通过类别占比。63 项逐条记录完整列于附录 B。

6.3 五阶段历史平台性能

版本	样例数	累计耗时	平均耗时	相对基线
XGrammar + 简单适配	50	116.873s	2.33746s	1.00×
纯 Python 优化版	50	32.821s	0.65642s	3.56×
混合稳定版	50	15.845s	0.31690s	7.38×
未加固纯 C++ 版	50	13.433s	0.26866s	8.70×
历史旧加固纯 C++ 版	50	13.664s	0.27328s	8.55×（历史旧加固版）

纯 Python 优化版中位数为 0.658s，p95 约 0.6962s，最小值 0.585s，最大值 0.708s；相对基线耗时降低 71.92%，50 个案例均取得加速。历史旧加固版最小值为 0.223s、最大值为 0.310s；相对基线平均耗时降低 88.31%，50 个案例全部取得加速。

上述逐例平台数据来自公开样例，检查器在公开标注的首错位置输出错误并终止，因此主要反映官方公开错误负载。合法长程序的完整流式行为另由项目语料、180 个随机合法程序、多分片测试和真实协议测试覆盖。混合稳定版与未加固纯 C++ 版只有已保存的 50 例平均值，累计耗时按 50 × 平均耗时 计算。

6.4 内部原生化版本的冷进程辅助数据

本队迭代版本	p50	p95	作用
混合稳定版	69.60ms	88.01ms	建立 C++ 协议、解码和语法入口，语义由 Python worker 执行
未加固纯 C++ 版	45.98ms	59.84ms	完成语义热路径 C++ 化
历史旧加固纯 C++ 版	53.49ms	71.34ms	增加多行、lambda、接口、重载和作用域检查

历史旧加固版以 1.72% 的平台平均耗时增幅换取更完整的隐藏样例式覆盖，并在当时口径下保持相对基线 8.55×（历史旧加固版）的加速。

6.5 与类似方法和参考工作的对比

本节将“平台实测”和“文献能力分析”分开。平台速度倍数只用于实际运行过的五阶段版本；论文与外部项目采用定性能力对比，不推导跨硬件、跨任务的速度结论。

方案	主要目标	增量语法	类型/作用域	未完成前缀	泛型/lambda	运行方式	本文证据类型
竞赛配套标准 typechecker	完整程序解析与语义检查	否	是	需要完整结构，不能直接接受多数未闭合前缀	依竞赛子集实现	Lark 完整解析与检查	离线 oracle、早期完整重检路径
XGrammar v0.2.1 [4]	结构化生成中的形式语法约束	是	否	维护语法可行前缀	只覆盖其语法形式	GrammarCompiler / GrammarMatcher 状态推进	本项目运行时实测
每 token 完整重检基线	将完整 typechecker 用于前缀检查	弱，每轮重新开始	是	需要额外的补全/容错策略	取决于完整 typechecker	每轮扫描、Lark 解析和类型检查	本项目基线实测
Koo 等的自动机约束方法 [1]	处理 tokenizer 与形式语言约束的对齐	是	论文重点为形式语言约束	支持正则及确定性上下文无关语言前缀	未针对仓颉类型语义	自动机编译与状态推进	文献分析
DOMINO [2]	子词对齐、低侵入的约束生成	是	论文重点为形式语言约束	支持子词对齐的约束前缀	未针对仓颉类型语义	预计算、speculative decoding 与约束解码	文献分析
类型约束代码生成 [3]	在生成中利用类型系统降低编译错误	论文包含 prefix automata	是	在形式化语言和 TypeScript 扩展中处理类型可达性	能力随论文模型与 TypeScript 扩展而定	prefix automata、可居住类型搜索	文献分析
本作品	仓颉赛题子集的语法和语义前缀检查	是	是	稳定词法事件、partial probe 与不可恢复错误判定	支持赛题三档用例涉及形式	XGrammar + 本队缓存模型/活动语句 C++ 语义探测	平台、公开样例、随机与分片实测

标准 typechecker 对完整程序提供语义 oracle，面对未闭合表达式时需要额外的前缀容错；每轮重新解析会重复处理历史字符。XGrammar 高效维护语法前缀状态，其职责范围不包含仓颉类型、作用域和调用匹配。本作品复用 XGrammar 的通用语法自动机，在其上增加面向仓颉赛题子集的增量语义状态、PartialLexeme 保守处理和不可恢复错误判定。该组合同时保留语法约束、语义首错和合法未完成前缀三类需求。

7. 正确性与鲁棒性测试

7.1 验证方法

测试证据按 oracle、版本和可支持结论分层：决赛平台负责最终得分与判定；平台列出的计时负责最终外部耗时；Behavioral Context Audit 负责校准决赛 API 行为；官方接受程序的前缀语料用于审计早拒；历史公开样例、随机程序、多分片和 sanitizer 只保留为相应阶段的回归证据。完整程序 typechecker 只负责开发期完整程序标签，不能替代逐 token 首错的官方判定。随机非法程序没有组委会提供的精确首错标签，因此本文不将其计入“精确首错”统计。

多维验证覆盖精确首错、合法前缀、随机变异与运行安全

所有结果来自项目记录的回归、随机生成器、真实协议和 sanitizer 验证。

精确首错	公开错误样例	50 / 50	检查首个不可恢复 token
固定语料	竞赛配套 + 项目语料	45 / 45 + 57 / 57	覆盖语义回归与项目维护案例
分片一致性	原生固定分片	66 / 66 × 4	逐字节、随机、cl100k、整段结果一致
随机稳健性	隐藏样例式生成	360 / 1800 / 0	程序数 / 分片执行数 / 失败数
合法前缀	180 个合法程序	0 误报	全部观测前缀保持可续写
运行安全	真实协议 + ASan/UBSan	48 / 48; 24 × 5	stdin/stdout 路径与内存安全检查

验收标准：错误首错准确，合法程序的每一个已观测前缀保持零误报。

7.2 分层验收结果

测试类别	Oracle / 期望来源	验证内容	结果	可支持的结论
决赛平台评分	组委会最终评测	隐藏用例总体得分与提交判定	63.00 / WA	最终版本最直接的外部正确性结果
决赛平台计时	最终结果页	页面列出的得分条目耗时	63 条 / 平均 0.26956s	只支持这 63 条记录的耗时结论
决赛 context 行为审计	官方 context/API 行为与生产 solution	106 个 context 成员的 A/B/C/D 探针矩阵	完成审计及定向校准	支持 first/last 与字段优先等定向修复；不表示所有差异均已消除
决赛合法前缀审计	1497 个官方接受程序	生产检查器是否在完整程序结束前 early fire	最终仍记录 38 次	定位剩余前缀风险，不能支持“最终零早拒”
数组 Alive 规则定向复验	官方完整程序检查器	237 个被延迟前缀接回各自原程序完整 tail	237/237 接受	支持该条 ValidSuffix 延迟规则的离线正向证据
历史公开样例 wrong/、wrong2/	当时公开锚点	精确首错 token	阶段记录各 50/50	仅为早期版本的历史证据；不冒充最终版本复验
历史旧加固版合法随机程序	竞赛配套完整 typechecker	最终合法且全部观测前缀无误报	180 程序 / 0 误报	历史随机证据，不直接替代最终版本官方结果
历史旧加固版非法随机变异	生成器 + 完整 typechecker + mutation_start	最终拒绝，且不在单点变异开始前拒绝	180 程序 / 0 失败	历史随机证据，未在未变异前缀提前误报
历史旧加固版多分片一致性	同一 UTF-8 字符流	byte、random、line、cl100k、whole 五种布局	360 × 5 = 1800 次 / 0 失败	历史分片一致性证据
历史旧加固版 ASan/UBSan	编译器 sanitizer	内存安全和未定义行为	24 程序 × 5 分片通过	历史运行安全证据

以下为初赛历史旧加固版证据：四个确定性随机种子 20260805、20260806、731991 和 991827 共生成 360 个程序，其中 180 个合法、180 个单点语义变异。每个程序使用五种分片布局执行，共 1800 次。所有程序的最终合法性与完整程序 oracle 一致；合法程序在全部观测前缀上未出现误报；非法程序未在 mutation_start 之前提前拒绝；不同分片布局的最终结论一致。覆盖家族包括多行泛型函数与调用、2-4 层嵌套 lambda、高阶泛型调用、重载与歧义方法引用、传递泛型接口继承、控制流、类/接口、注释、作用域隔离和关键字前缀标识符。

7.3 代表性复现命令

```
./build.sh
python3 benchmark/differential_check.py --solution ./solution
python3 benchmark/native_fragment_differential.py
python3 benchmark/native_context_differential.py
python3 -m unittest discover -s tests
python3 benchmark/hidden_semantic_fuzz.py --seed 20260805 --cases-per-family 12
python3 benchmark/hidden_semantic_fuzz.py --seed 424242 --cases-per-family 4 --solution ./solution
python3 benchmark/hidden_semantic_fuzz.py --seed 868686 --cases-per-family 2 --sanitize
python3 benchmark/production_benchmark.py --solution ./solution
```

上述命令用于仓库内复现，不会重新生成组委会的隐藏评分。最终 63.00 / WA 与 63 条耗时以平台结果页记录为准。

8. 为什么选择原生 C++ 与持续语义加固

8.1 团队的工程取舍

原生 C++ 路径把竞赛协议、token 解码、XGrammar 语法状态、增量词法和语义检查收敛到单一生产进程，避免每轮启动解释器或重复执行完整程序解析。最终版本还直接编译随包分发的 XGrammar C++ core，不需要 Python 绑定或 TVM FFI 运行依赖，使构建边界和提交物归属更清晰。

历史旧加固纯 C++ 版本曾在初赛平台口径下达到 0.27328s、相对简单适配基线 8.55×（历史旧加固版）。最终提交的 63 条平台记录平均 0.26956s，但样本集合不同，因此本节不据此重算加速倍数。最终版本的取舍重点是：在原生增量骨架上，用可定位的 context 行为修复与有证据的前沿覆盖提高场景覆盖，并用 G1/G4 消除长 block 和长标识符的规模悬崖。

8.2 加固重点降低的潜在风险

最终版本的加固覆盖以下组合性风险：

- 声明时序风险：同行 var/let 需要在同一活动语句内更新可见上下文，不能只依赖换行或分号后的全量收集。
- 控制流形态风险：循环体可以是裸标识符或无花括号语句，循环绑定和作用域判断不能只匹配块结构。
- 非代码文本污染风险：字符串和注释中的声明、字段和构造器文本必须在扫描前掩码，否则会形成伪语义事实。
- 字段初始化风险：var 字段的构造器初始化需要结合类模型、构造器体和非代码文本掩码共同判断。
- 调用边界风险：尾随空参数不能在闭合调用时重复挂接；min/max 泛型行为需要按官方 context API 的实测边界校准。
- 开放表达式风险：找不到短后缀只能得到 Unknown；开放数组延期规则需要离线完整 tail 的正向证据支撑，闭合调用的拒绝需要完整候选集耗尽。
- context 行为风险：JSON 的成员形态不等于官方读取/调用语义，需要区分 Array 属性化、实例字段优先和静态特殊行为。
- 规模退化风险：右递归 block 与超长标识符会放大 XGrammar 状态，必须采用语言等价或词法模式隔离的窄优化。
- 既有复杂结构风险：多行声明、嵌套 lambda、传递泛型接口、重载消歧和函数作用域隔离继续作为回归基线。

这些改动可以追溯到 cpp/native_semantic.cpp、cpp/call_frontier.cpp、cpp/continuation.cpp、cpp/solution.cpp、两份仓颉文法。第三方通用语法引擎和开发期 oracle 不计入上述团队贡献。

9. 已知边界与适用范围

本作品面向赛题指定的仓颉子集和组委会提供的 context，文中结论不外推到完整仓颉 1.0.0 语言。

最终检查器采用工程化保守判定，不对全部可能后缀执行显式枚举，也没有给出完整仓颉语言的形式化可达性证明。生产证明层目前只有开放数组元素的 Alive + ValidSuffix 与闭合调用候选耗尽的 Dead + ClosedWorldExhaustive 两条正式规则；OfficialAudit 仅保留为类型定义，当前没有生产规则返回它。

TokenEvent 当前负责稳定词法边界，语义检查主要结合完整可见源码、缓存声明模型、函数上下文和活动语句进行选择探测。完全由 TokenEvent 驱动的声明和表达式单次提交属于后续优化方向。

作用域能力以函数参数、函数体局部变量、函数间隔离、循环绑定和活动 lambda 上下文等已测试场景为主，文中结论不覆盖完整仓颉语言的全部嵌套块遮蔽、复杂闭包捕获和非常规生命周期组合。

结构提交点或上下文探测仍可能扫描较大范围源码，活动表达式 probe 也可能重新推导局部表达式。G1/G4 消除了两类已知规模悬崖，但不构成所有输入上的线性复杂度证明。

1497 项官方接受程序的最终前缀扫描仍记录 38 次 early fire，说明 postfix initializer、method reference 及少量 clamp、array-index、generic 前缀仍需进一步修复。决赛最终外部结果也是 63.00 / WA，不能把内部差分或历史公开回归解释为全部隐藏语义已经覆盖。

第 6.2 节只统计决赛页面列出的 63 条计时；第 6.3–6.4 节和附录 A 的性能结论只对应初赛历史 50 例在限定 Docker、Ubuntu 22.04、ARM 环境中的旧平台负载。历史公开两组各 50/50 也只代表当时的锚点，仓库内旧锚点与决赛口径不同，不作为最终版本的当前复验结论。

10. 第三方来源与原创贡献声明

10.1 可审计组件清单

版本与归属依据当前仓库 THIRD_PARTY_NOTICES.md、上游说明文件和生产构建脚本填写。

组件	版本/来源	当前用途	进入最 终版本 运行路 径	随包形式	归属结论
XGrammar C++ core	v0.2.1, commit 5b4e9ce9e72524037ae24ecd831b9b6604d2eb48, Apache-2.0	GBNF 编译和增量 语法匹配	是	third_party/xgrammar_core/. 上游内容未作功能性修改	通用 matcher 非团队原创；仓 库 GBNF 与接入代码为团队 适配
DLPack / PicoJSON	随 XGrammar core 分发，许可证见对应文件	XGrammar core 的 头文件依赖	是	XGrammar 子目录内	第三方
tiktoken	0.13.0, MIT	生成 cl100k_base.bin 时提供编码定义	否	不分发 tiktoken 源码/库；随包提 供由其编码定义生成的 assets/cl100k_base.bin.xz	编码定义非团队原创；二进 制表生成工具为团队实现
TVM FFI	Apache-2.0	历史 Python/XGrammar 绑定相关基础能力	否	最终版本 C++ core 构建不包含其 绑定	非团队原创，且不是当前生产 运行依赖
竞赛官方 context 与 提交适配	决赛官方 context 为原始来源；提交 context.json 经团队行为校准	生成预定义符号模 型	生成输 入	context.json 经结构序列化形成 generated/context.bin	原始数据归官方； first/last 字段化等行为为校 准、转换工具与原生加载为团 队实现
竞赛配套 typechecker	竞赛公开参考实现	开发期差分、随机 标注与 oracle	否	不进入当前提交包的生产构建/运 行	上游与本地开发期适配均不计 团队原创

当前 build.sh 直接编译 third_party/xgrammar_core/，不下载或动态链接 Python XGrammar/TVM FFI。tiktoken 只用于已生成词表的来源定义；generated/cl100k_base.bin 与 generated/context.bin 在运行时由 C++ 直接读取。

10.2 第三方能力、本队适配与独立实现边界

下面的审计表把“独立实现”“适配集成”和“第三方能力”分开，并给出当前仓库内可定位证据。

归 属 类 别	工作项	实现路径	可声明范围
团 队 独 立 实 现	C++ 竞赛入口、协议输出与首错锁存	cpp/solution.cpp	可作为团队原 创工程实现

团队独立实现	token ID 本地查表解码	cpp/solution.cpp、 tools/generate_cl100k_table.py、 generated/cl100k_base.bin	工具和查表接入原创；编码定义归 tiktoken
团队独立实现	IncrementalLexer、TokenEvent、PartialLexeme	cpp/native_semantic.h、 cpp/native_semantic.cpp	可作为团队原创实现
团队独立实现	NativeSemanticChecker、 IncrementalSemanticEngine、Model、 DeclarationSnapshot、FunctionContext、 ExpressionTyper、ActiveStatementCache	cpp/native_semantic.cpp	可作为团队原创增量语义实现
团队独立实现	同行声明、循环体裸标识符、非代码文本掩码、字段初始化、尾随空参数、min/max 校准	cpp/native_semantic.cpp	最终版本的真实语义贡献
团队独立实现	context 单源加载、行为校准、Array 首末属性与字段优先解析	tools/generate_context_table.py、 cpp/native_semantic.cpp、context.json	可作为团队原创适配和语义实现
团队独立实现	lambda twin、逐重载调用前沿与证明携带决策	cpp/native_semantic.cpp、 cpp/call_frontier.*、cpp/continuation.*	可作为团队原创前缀判定实现；BFS 见证不声明为生产裁决
团队独立实现	标识符文法有限商与可回滚单步试探	cpp/solution.cpp	有限商接入为团队实现；底层 matcher 归 XGrammar
团队独立实现	context API 差分、综合案例和验证工具	benchmark/context_api_differential.py、 tools/run_comprehensive_cases.py、 tools/behavioral_context_audit.py	可作为团队原创验证工程
团队适配与集	仓颉赛题子集 GBNF 与 block 语句表平坦化	grammar/cangjie.gbnf、 grammar/cangjie_token.gbnf	文法适配为团队工作；通用 GBNF 引擎非原创

成

团队适配与集成	XGrammar C++ API、context/token 二进制化和构建链路	cpp/solution.cpp、tools/、build.sh	接入与构建为团队工作
第三方基础能力	XGrammar 通用 GrammarCompiler / GrammarMatcher	third_party/xgrammar_core/	不声明原创
第三方基础能力	TVM FFI、tiktoken 编码定义、竞赛配套 typechecker	vendor/tvm_ffi/、生成期 API、third_party/cangjie_typechecker/	均不声明原创

团队贡献的核心不是重新发明通用语法自动机，而是把仓颉赛题的 token/context 数据、逐 token 协议、前缀词法状态和语义规则组合成可运行、可验证的 C++ 检查器。任何无法指向代码、提交或测试记录的能力和性能数字，本文均不作为团队成果声明。

10.3 竞赛配套 typechecker 的开发期用途与提交边界

third_party/cangjie_typechecker/ 来源于竞赛配套公开参考实现，只用于开发期完整程序解析、差分和随机样本标注。其上游实现以及本地为了测试范围所做的适配，均不属于本队原创成果，也不计入本队原创代码量。

最终版本生产程序的编译、链接与运行不调用该 typechecker。正式运行路径由本队 C++ 入口和语义实现、本队仓颉 GBNF 与生成工具，以及第三方 XGrammar C++ core 构成。开发期 oracle 只能说明被测完整程序的标签或差分结果，不能替代公开样例的逐 token 首错结果；因此第 7 章将最终平台、决赛行为审计和历史随机证据分开报告。

10.4 最终提交框架清单

最终提交框架按生产、第三方和开发验证边界列示：

路径或材料类别	最终版本角色	原创/归属说明
cpp/	生产 C++ 入口、增量词法与语义检查	团队独立实现
grammar/	仓颉赛题子集 GBNF	团队适配与维护
tools/	context/token 生成与综合验证工具	团队独立实现
assets/	随包压缩 token 表，供离线构建时解压	基于 tiktoken 编码定义生成；生成与解压链路为团队实现
generated/	context 与 cl100k 二进制表	由团队工具基于经行为校准的提交 context 与 tiktoken 编码定义生成
build.sh	当前生产构建入口	团队独立实现
third_party/xgrammar_core/	生产语法引擎 core	XGrammar v0.2.1 第三方源码，未作功能性修改
third_party/cangjie_typechecker/	开发期差分材料，不参与生产路径	竞赛参考实现及本地测试适配，均不计原创
benchmark/、tests/、baseline_results/	差分、回归与历史证据记录	团队验证工程；其中 oracle 来源另行归属
THIRD_PARTY_NOTICES.md	第三方来源与许可证说明	合规说明

TVM FFI 和 tiktoken 不属于团队原创。最终版本运行时不依赖二者：TVM FFI 不进入当前 C++ core 路径，tiktoken 只在词表生成来源中使用。

11. 参考文献

1. Terry Koo, Frederick Liu, Luheng He. [Automata-based constraints for language model decoding](#). COLM 2024.
2. Luca Beurer-Kellner, Marc Fischer, Martin Vechev. [Guiding LLMs The Right Way: Fast, Non-Invasive Constrained Generation](#). ICML 2024, PMLR 235:3658–3673.
3. Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, Martin Vechev. [Type-Constrained Code Generation with Language Models](#). PACMPL 9 (PLDI), 2025.
4. MLC-AI. [XGrammar v0.2.1](#), Apache License 2.0.
5. 仓颉编程语言社区. [仓颉编程语言 1.0.0 文档](#).
6. Apache Software Foundation. [Apache TVM FFI Documentation](#).
7. OpenAI. [tiktoken](#), MIT License.
8. 竞赛配套项目. [cangjie-fragment-checker](#).

12. 结论

本作品以 XGrammar 通用语法引擎和本队仓颉适配为基础，完成了从逐 token 完整重检到原生 C++ 增量前缀语义检查的演进。最终版本的生产路径统一处理 token 查表解码、标识符有限商、XGrammar 语法状态、稳定词法事件、未完成词素、声明快照、活动上下文、逐重载调用前沿、证明携带决策与首错锁存；构建直接使用 XGrammar v0.2.1 C++ core，不依赖 Python 绑定或 TVM FFI。

决赛最后平台结果为 63.00 / WA。页面列出的 63 条计时累计 16.982s、平均 0.26956s/条、中位数 0.270s。这是最终版本最直接的外部结论；历史公开回归、context 行为审计和本地随机测试只用于解释实现与回归，不能替代该结果。

最终版本可以审计的团队贡献包括 C++ 竞赛入口、token 查表解码、IncrementalLexer、TokenEvent/PartialLexeme、原生增量语义模型、经行为校准的 context 运行时单源加载、Array 首末属性、字段优先解析、lambda twin、单步可恢复性、CallFrontier、ContinuationProof/DecisionLedger、block 文法平坦化和标识符有限商。XGrammar 通用 matcher、TVM FFI、tiktoken 编码定义和竞赛参考 typechecker 均已明确排除在原创声明之外。

最终结果也暴露出明确边界：1497 项官方接受程序的最终前缀扫描仍有 38 次 early fire，隐藏评测仍为 WA。后续工作应优先最小化 postfix initializer、method reference 与少量泛型/索引前缀反例，并只以可复验的延期证据或硬边界穷尽证明扩展生产规则。初赛旧加固纯 C++ 版本的 8.55×（历史旧加固版）和附录 A 仅保留为架构历史，不作为最终版本的直接性能倍数。

附录 A：初赛历史逐例平台性能数据

本附录完整保留初赛旧版本的逐例平台性能证据。下表单位均为秒，单例提速倍数按“XGrammar + 简单适配耗时 ÷ 历史旧加固纯 C++ 耗时”计算。该表不能代表最终版本的直接性能。

案例	XGrammar + 简单适配	纯 Python 优化版	历史旧加固纯 C++	基线→历史旧加固提速
err_undefined	3.121	0.708	0.294	10.62×
err_assign_let	3.706	0.702	0.310	11.95×
err_arity	3.015	0.680	0.292	10.33×
err_if_not_bool	2.313	0.658	0.277	8.35×
err_break	2.891	0.669	0.289	10.00×
err_type_mismatch	2.604	0.669	0.287	9.07×
err_continue_outside_loop	2.604	0.670	0.293	8.89×
err_return_type_mismatch	0.862	0.585	0.223	3.87×
err_duplicate_var	0.916	0.596	0.224	4.09×
err_arraylist_toarray_assign	2.468	0.663	0.284	8.69×
err_unary_minus_non_numeric	2.196	0.648	0.277	7.93×
err_eq_incomparable	2.322	0.658	0.269	8.63×
err_rel_mixed_numeric	2.057	0.647	0.265	7.76×
err_mod_non_int64	2.008	0.645	0.263	7.63×
err_range_non_int	2.213	0.653	0.265	8.35×
err_for_not_iterable	2.133	0.646	0.262	8.14×
err_for_pattern_map_bad	2.333	0.664	0.274	8.51×
err_array_index_not_int64	2.991	0.689	0.288	10.39×
err_array_fill_type	2.753	0.676	0.283	9.73×

err_string_contains_arg	2.525	0.659	0.280	9.02×
err_ctor_call_mismatch	2.678	0.672	0.280	9.56×
err_generic_arity	2.476	0.666	0.282	8.78×
err_interface_not_implemented	0.925	0.593	0.227	4.07×
err_interface_sig_mismatch	0.942	0.593	0.223	4.22×
err_no_member	2.216	0.646	0.272	8.15×
err_unknown_named_arg	2.380	0.658	0.272	8.75×
err_index_non_array	1.981	0.645	0.262	7.56×
err_unary_not_non_bool	1.910	0.644	0.259	7.37×
err_while_not_bool	2.122	0.645	0.265	8.01×
err_arraylist_add_type	2.290	0.656	0.278	8.24×
err_hashmap_key_type	2.283	0.657	0.280	8.15×
err_bound_var_mismatch	2.062	0.644	0.266	7.75×
err_rel_unordered	2.091	0.647	0.265	7.89×
err_interface_as_value	1.829	0.648	0.264	6.93×
err_arith_non_numeric	2.618	0.665	0.281	9.32×
err_lambda_arg_arity_explicit	2.866	0.670	0.276	10.38×
err_lambda_return_type_explicit	2.465	0.660	0.273	9.03×
err_lambda_zero_body_explicit	2.006	0.643	0.265	7.57×
err_lambda_hof_explicit	3.725	0.694	0.295	12.63×
err_lambda_interface_callback_explicit	2.919	0.698	0.302	9.67×
err_lambda_in_class_static_explicit	2.087	0.657	0.274	7.62×
err_lambda_param_narrow_explicit	2.419	0.655	0.275	8.80×
err_lambda_infer_ambiguous_1	2.378	0.657	0.270	8.81×
err_lambda_infer_ambiguous_2	2.991	0.668	0.286	10.46×
err_lambda_infer_wrong_return_1	1.996	0.636	0.265	7.53×
err_lambda_infer_wrong_return_2	3.093	0.669	0.282	10.97×
err_lambda_infer_collection_1	2.201	0.658	0.277	7.95×
err_lambda_infer_collection_2	2.384	0.664	0.278	8.58×
err_lambda_infer_class_helper	2.139	0.657	0.278	7.69×
err_lambda_infer_interface_helper	2.370	0.671	0.293	8.09×

附录 A 汇总

指标	XGrammar+ 简单适配	纯 Python 优化版	历史旧加固纯 C++
样例数	50	50	50
累计耗时	116.873s	32.821s	13.664s
平均耗时	2.33746s	0.65642s	0.27328s
中位数	2.3275s	0.658s	0.2765s
p95（线性插值）	3.1084s	0.6962s	0.29455s
最小值	0.862s	0.585s	0.223s
最大值	3.725s	0.708s	0.310s

附录 B：决赛最终提交平台数据

数据来自决赛平台最终提交页，最后提交时间为 2026-08-19 14:50:09，得分 63.00，判定 WA。平台页面列出以下 63 个计时条目；本附录保持用户提供的名称、顺序和秒数，不用历史公开样例替换。平均耗时只针对这些已列出条目计算。

序号	案例	用时	序号	案例	用时
1	err_array_ctor_shape	0.283s	2	err_array_slice_index	0.299s
3	err_hashmap_replace_value	0.294s	4	err_dual_iface_missing	0.244s
5	err_string_replace_arg	0.273s	6	err_string_empty_to_int	0.298s
7	err_hashmap_contains_key	0.291s	8	err_arraylist_get_optional	0.275s
9	err_string_indexof_optional	0.274s	10	err_array_indexof_elem	0.265s
11	err_optional_is_some_bool	0.257s	12	err_abs_to_string	0.260s
13	err_abs_float_family	0.260s	14	err_min_mixed_family	0.264s
15	err_println_overload	0.274s	16	err_stack_add_elem	0.278s
17	err_stack_peek_optional	0.264s	18	err_stack_remove_optional	0.288s
19	err_deque_add_first_elem	0.274s	20	err_deque_remove_last_int	0.273s
21	err_deque_add_last_drift	0.315s	22	err_stack_deque_add_first	0.273s
23	err_deque_toarray_join	0.248s	24	err_stack_ctor_cap	0.255s
25	err_stack_static_add	0.262s	26	err_deque_reserve_arg	0.258s
27	err_stack_empty_int	0.267s	28	err_deque_clear_unit	0.253s
29	err_arraylist_get_throw_str	0.273s	30	err_string_indexof_arith	0.256s
31	err_hashmap_contains_int_key	0.273s	32	err_max_clamp_family	0.273s

33	err_min_if_condition	0.252s	34	err_abs_bool_helper	0.248s
35	err_stack_size_call	0.257s	36	err_deque_capacity_string	0.260s
37	err_array_first_optional	0.254s	38	err_array_last_abs	0.259s
39	err_deque_fill_family	0.272s	40	err_array_swap_list_get	0.271s
41	err_hashset_add_if_absent	0.252s	42	err_string_compare_indexof	0.242s
43	err_list_deque_add_last	0.268s	44	err_hashmap_remove_keys	0.252s
45	err_lambda_pair_arity	0.251s	46	err_lambda_param_narrow	0.254s
47	err_lambda_iface_callback	0.307s	48	err_lambda_not_function	0.254s
49	err_lambda_opt_thunk	0.274s	50	err_lambda_deque_return	0.283s
51	err_lambda_stack_nested	0.291s	52	err_lambda_max_by	0.281s
53	err_lambda_clamp_pipeline	0.293s	54	err_lambda_pick_zero	0.284s
55	err_infer_witness_trio	0.266s	56	err_infer_forin_generic	0.243s
57	err_infer_min_max_fuse	0.275s	58	err_infer_map_lookup	0.274s
59	err_infer_list_putget	0.274s	60	err_infer_nested_container	0.295s
61	err_infer_stack_ctor	0.266s	62	err_infer_map_contains	0.270s
63	err_infer_hof_param	0.266s			

附录 B 汇总

指标	结果
最后提交时间	2026-08-19 14:50:09
得分 / 判定	63.00 / WA
平台列出的计时条目	63
累计耗时	16.982s
算术平均耗时	0.2695556s（正文取 0.26956s）
中位数	0.270s
p95（线性插值）	0.2977s
最小值	0.242s（err_string_compare_indexof）
最大值	0.315s（err_deque_add_last_drift）